

多Agent协作与调度系统

面试准备文档

项目类型	Agent 大模型工程师面试项目
技术栈	Python + asyncio + Pydantic + aiohttp
核心模型	Qwen-Max (DashScope)
GitHub 仓库	github.com/tanghui2/multi-agent-orchestration
文档版本	v1.0

一、项目使用场景

1.1 项目定位

多Agent协作与调度系统是一个基于 Python asyncio 的 Agent 协作框架，通过编排器（Orchestrator）实现任务规划、依赖管理和多 Agent 协同工作。系统支持三种内置 Agent（调研/编码/审查），通过消息总线实现 Agent 间通信，通过上下文管理器实现项目级隔离和跨 Agent 共享。核心特性包括：LLM 驱动自动任务分解、拓扑排序的依赖执行、发布/订阅模式的消息传递。

1.2 典型使用场景

- 自动化软件开发流水线：Researcher 调研技术方案 → Coder 实现代码 → Reviewer 审查质量，形成完整的开发闭环。
- 智能数据分析：Researcher 收集数据 → Analyst 分析趋势 → Coder 生成报表，实现从数据到洞察的自动化。
- 复杂问题求解：Planner 分解复杂问题 → 多个 Specialist Agent 并行处理 → Orchestrator 汇总结果。
- 代码质量保障：Coder 实现功能 → Reviewer 代码审查 → 修复 → 二次审查，保证交付质量。
- 知识库构建：Researcher 收集知识 → Analyst 结构化整理 → Coder 实现检索接口，构建可查询的知识库。

1.3 适用人群与价值

该系统适用于需要让多个 Agent 协同完成复杂任务的场景，为开发者提供了完整的多 Agent 工程化范式：包括任务规划、依赖管理、消息传递、上下文隔离等。对于企业而言，可作为内部自动化流水线的智能引擎；对于开发者而言，是理解多 Agent 协作架构的最佳实践样本。

二、项目整体框架

2.1 分层架构

项目采用分层架构设计，自下而上分为：配置层、基础设施层、协议层、上下文层、Agent 层、编排层。

层级	模块	职责
配置层	config.py / config.toml	单例模式管理 LLM、编排器、Agent、上下文等配置

层级	模块	职责
基础设施层	models.py / llm.py / logger.py	Pydantic 数据模型、LLM 封装（含重试）、结构化日志
协议层	protocol/*.py	消息总线（发布/订阅）、任务队列（优先级+依赖）
上下文层	context/*.py	项目级隔离、Agent 级上下文、全局共享记忆
Agent 层	agents/*.py	BaseAgent 基类、Researcher/Coder/Reviewer 实现
编排层	orchestrator/*.py	任务规划、Agent 调度、依赖执行、生命周期管理

2.2 核心组件详解

- Orchestrator（编排器）

系统的核心调度中心，负责任务规划、Agent 注册、任务分配和执行控制。通过 plan_and_execute() 实现 LLM 驱动的任务自动分解；通过 _topological_sort() 实现依赖排序执行；通过异步超时机制控制任务执行时间。

- MessageBus（消息总线）

基于发布/订阅模式的异步消息传递系统。支持三种消息类型：request（请求-响应）、response（响应）、broadcast（广播）。使用 asyncio.Queue 实现消息队列，支持消息历史查询和订阅者动态增删。

- TaskQueue（任务队列）

基于优先级的任务调度队列，支持任务依赖管理。使用 heapq 实现优先级排序，支持四个优先级（critical/high/medium/low）。通过 _dependencies_met() 检查任务依赖是否满足，确保执行顺序正确。

- ContextManager（上下文管理器）

支持项目级隔离和 Agent 级上下文的双重上下文管理。项目级上下文保存全局历史消息，Agent 级上下文保存专属交互历史。同时提供全局共享记忆，支持跨 Agent 的知识传递。

- BaseAgent（Agent 基类）

采用模板方法模式设计的 Agent 抽象基类。定义 run() 生命周期方法，子类实现 execute() 核心逻辑。内置协作协议：request_collaboration()、respond_to_request()、broadcast_message()。支持状态管理（idle/busy/offline）和消息自动订阅。

2.3 核心执行流程

步骤 1 创建编排器：实例化 Orchestrator，配置 LLM、消息总线、任务队列等核心组件。

步骤 2 注册 Agent：将 Researcher、Coder、Reviewer 等 Agent 注册到编排器，建立能力映射。

步骤 3 任务规划：调用 `plan_and_execute()`，LLM 自动将目标分解为 3-5 个子任务，明确依赖关系。

步骤 4 依赖排序：`_topological_sort()` 对任务进行拓扑排序，确保依赖任务先执行。

步骤 5 Agent 分配：`_assign_agent()` 根据任务类型匹配 Agent 能力（如 research 类型分配给 ResearcherAgent）。

步骤 6 任务执行：`run()` 方法执行任务，构建上下文 → 调用 `execute()` → 返回结果，带超时保护。

步骤 7

状态更新：任务完成后更新状态（completed/failed），结果写入上下文供后续任务使用。

2.4 协作协议

- 请求-响应模式：Agent A 通过 `request_collaboration()` 发送请求给 Agent B，等待 B 响应后继续执行。使用 `asyncio.Future` 实现同步等待，支持超时控制。
- 广播模式：Agent 通过 `broadcast_message()` 向所有 Agent 发送消息，用于通知或状态同步。订阅者自动收到消息并触发 `on_broadcast()` 回调。
- 任务传递：前序任务的输出（`output_data`）作为后续任务的输入（`input_data`），形成数据流水线。依赖关系通过 `task.dependencies` 显式声明。

三、项目过程中遇到的问题

3.1 任务依赖的环形依赖问题

问题描述：多个任务之间可能存在环形依赖（如 A 依赖 B，B 依赖 A），导致执行流程卡死。

解决方案：实现 Kahn 算法拓扑排序，检测并处理环形依赖：如果排序后的任务数与总数不符，回退到原顺序执行，并记录警告日志。

收获：掌握了图算法在工程中的应用，理解了环形依赖检测是任务调度系统的基本能力。

3.2 Agent 能力匹配的准确性问题

问题描述：简单的角色匹配 (researcher/coder/reviewer) 不够灵活，无法支持自定义 Agent 和复杂任务类型。

解决方案：引入 capabilities 列表机制，每个 Agent 声明自己的能力标签；_assign_agent() 通过能力标签匹配，支持模糊匹配和回退策略。

收获：理解了"能力标签"比"角色分类"更灵活的设计思想，开放-封闭原则的实际应用。

3.3 异步消息总线的顺序保证

问题描述：多个 Agent

同时发送消息时，消息的处理顺序可能不一致，导致依赖上下文的逻辑出错。

解决方案：使用 asyncio.Queue 保证消息的 FIFO 顺序；每条消息携带 timestamp 标识创建时间；消息历史支持按时间排序查询。

收获：掌握了异步系统中的顺序保证技术，理解了消息队列在并发场景下的重要性。

3.4 Agent 状态管理的一致性

问题描述：Agent 在执行任务过程中可能异常退出，导致状态未正确更新（仍为 busy），影响后续任务分配。

解决方案：在 BaseAgent.run() 的 finally 块中强制设置状态为 IDLE；使用 try-except 捕获所有异常，确保状态一定能被重置。

收获：理解了"状态清理"是异步编程的关键细节，finally 块的重要性。

3.5 超时控制与资源清理

问题描述：长时间运行的 LLM 调用或外部 API 可能导致任务超时，但 Agent 和编排器的状态无法正确恢复。

解决方案：使用 asyncio.wait_for() 包裹任务执行，超时后捕获 TimeoutError；在 finally 块中清理资源；心跳机制定期检查超时任务。

收获：掌握了 asyncio 超时控制的正确用法，理解了"超时不等于取消"的区别。

3.6 上下文隔离与共享的平衡

问题描述：完全隔离会导致 Agent 间无法共享信息，完全共享会导致上下文混乱和 Token 超限。

解决方案：采用"项目级共享 + Agent 级隔离"的混合模式：项目内所有 Agent 共享历史消息，每个 Agent 还有独立的专属上下文；通过 max_history 限制历史消息数量。

收获：理解了隔离与共享的工程权衡，掌握了混合隔离模式的设计方法。

四、面试官可能提出的问题

4.1 架构设计类

Q1：为什么选择 Orchestrator 模式，而不是让 Agent 直接对话？

Orchestrator 模式的优势：① 集中控制：编排器统一管理任务分配和执行顺序，避免 Agent 间的混乱协调；② 可观测性：所有任务和 Agent 状态集中在编排器中，便于监控和调试；③ 可扩展：新增 Agent 类型只需注册到编排器，无需修改现有 Agent。Agent 直接对话的缺点是缺乏全局视角，容易出现死锁或资源争抢。

Q2：为什么用模板方法模式设计 BaseAgent？

模板方法模式保证了 Agent 生命周期的一致性：run() 方法定义了「构建上下文 → 执行 → 处理结果」的标准流程，子类只需实现 execute() 核心逻辑。好处：① 统一的错误处理和状态管理；② 易于添加横切关注点（如日志、指标）；③ 符合"开放扩展、封闭修改"原则。

Q3：消息总线为什么用发布/订阅模式？和直接方法调用相比有什么优势？

发布/订阅模式的优势：① 解耦：发送方不需要知道接收方的存在；② 广播：一条消息可以被多个订阅者接收；③ 可扩展：新增订阅者不影响发送方。直接方法调用的缺点是紧耦合，不支持广播，且在多 Agent 场景下需要维护引用关系。

4.2 技术细节类

Q4：拓扑排序是如何实现的？如何处理环形依赖？

使用 Kahn 算法实现拓扑排序：① 构建邻接表和入度数组；② 将入度为 0 的任务加入队列；③ 依次取出队列头部任务，更新邻居入度，入度变为 0 的加入队列。环形依赖处理：如果排序后的任务数与总数不符，说明存在环，回退到原顺序执行。

Q5：任务队列的优先级是如何实现的？

使用 `heapq` 实现最小堆。每个任务的权重由 `Priority -> 数值映射` 决定 (`critical=0, high=1, medium=2, low=3`)，权重越小越优先。堆中元素为 (`weight, created_at, task_id`)，保证同优先级任务按创建时间排序。

Q6: 上下文隔离的具体策略是什么？

采用双层隔离：① 项目级隔离：每个项目有独立的上下文，保存该项目所有 Agent 的交互历史；② Agent 级隔离：每个 Agent 在每个项目中还有独立的专属上下文。同时提供全局共享记忆，通过 `shared_memory` 传递关键信息。通过 `max_history` 限制消息数量，防止 Token 超限。

Q7: LLM 驱动的任务规划是如何工作的？

`plan_and_execute()` 调用 LLM 的 `ask_json()` 方法，传入目标描述，要求 LLM 返回 JSON 数组格式的任务分解。每个任务包含 `title`、`description`、`task_type`、`depends_on` 四个字段。解析 LLM 返回的 JSON，创建 Task 对象并添加到任务队列。失败时回退为单一任务。

Q8: 异步超时控制的实现？

使用 `asyncio.wait_for(task, timeout=300)` 包裹 `Agent.run()` 调用。超时后 `asyncio.wait_for` 抛出 `TimeoutError`，捕获后设置任务状态为 `failed`，错误信息为“执行超时”。注意 `asyncio.wait_for` 会取消内部任务，需要正确处理 `CancelledError`。

4.3 工程实践类

Q9: 如何保证多 Agent 系统的稳定性？

三层保障：① 超时控制：每个任务有执行超时，超时后强制终止；② 状态管理：finally 块保证 Agent 状态一定能被重置；③ 心跳检查：编排器定期检查 Agent 状态，清理异常 Agent。另外，LLM 调用使用 `tenacity` 重试机制，自动处理网络异常和限流。

Q10: 如何评估多 Agent 协作的效果？

从三个维度评估：① 任务完成率：规划的任务中成功完成的比例；② 依赖满足率：任务依赖正确执行的比例；③ 协作效率：完成目标所需的总步数和 LLM 调用次数。可通过对比不同架构（单 Agent / 多 Agent）的效果来评估。

Q11: 如果要支持更多类型的 Agent，需要做哪些改动？

只需两步：① 继承 `BaseAgent`，实现 `execute()` 方法和 `_get_system_prompt()`；② 在 `__init__` 中声明角色和能力。编排器会自动根据能力标签进行任务分配，无需修改编排器代码。这符合“开放扩展、封闭修改”的设计原则。

Q12: 项目有哪些可改进的方向？

① 动态 Agent 发现：Agent 可以注册自己的能力，编排器根据任务需求自动发现合适的 Agent；② 任务回滚：支持任务失败后的回滚和重试；③

并行执行：支持无依赖任务的并行执行，提升效率；④
持久化状态：编排器状态持久化到数据库，支持断点续执行；⑤ 多模态支持：扩展 Agent
支持图像、音频等多模态输入输出。

4.4 扩展问题

Q13：如果要把这个系统应用到生产环境，需要做哪些改造？

① 消息持久化：消息总线增加 Redis 或 Kafka 作为消息存储；② 分布式部署：编排器和 Agent
支持多实例部署，通过分布式锁协调；③ 监控告警：增加 Prometheus 指标和 Grafana
仪表盘；④ API 网关：提供 RESTful/gRPC 接口；⑤ 权限控制：基于角色的访问控制；⑥
审计日志：记录所有操作和决策过程。

Q14：多 Agent 协作和微服务架构有什么异同？

相同点：都是分布式、独立部署、通过消息通信。不同点：① 多 Agent 有自主决策能力（基于
LLM），微服务是确定性的；② 多 Agent 可以动态分工和协作，微服务职责固定；③ 多 Agent
需要额外的信任和安全机制。可以借鉴微服务的设计模式（服务发现、负载均衡、熔断降级）来
增强多 Agent 系统。

Q15：这个项目最难的地方是什么？

不是单一技术点，而是「协作协议设计」：① 异步消息的顺序保证和一致性；② Agent
间的依赖关系管理，避免死锁；③ 上下文传递的正确性，防止信息丢失或污染；④
超时和异常的处理，保证系统不会僵死；⑤ 可观测性设计，在复杂的多 Agent
交互中快速定位问题。

附录：项目技术栈速查

类别	技术 / 库	用途
语言	Python 3.11+	主开发语言，异步生态成熟
AI 模型	Qwen-Max (DashScope)	任务规划、决策、代码生成
数据模型	Pydantic	Schema 定义、数据校验、序列化
异步框架	asyncio	协程并发、消息队列、任务调度
HTTP 客户端	aiohttp	异步 HTTP 请求 (LLM API 调用)
重试	tenacity	指数退避、异常类型过滤重试
配置	toml	结构化配置文件管理
日志	logging	结构化日志、多级别输出

类别	技术 / 库	用途
数据结构	heapq	优先队列实现（任务调度）

提示：面试时重点突出「为什么这样设计」而非「实现了什么」，展现工程权衡能力。对于每个技术选型，准备好备选方案和取舍理由。